SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Operating Systems **COURSE CODE: CSA04**

COURSE OUTCOME COVERED

CO5: *Analyze the components and functionalities of Linux operating systems and virtualization environments to enhance their operational efficiency*

Assessment Tool Weightage: 35%

Dr J Priskilla Angel Rani

QUESTIONS: 10 **Total Marks:50**

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I Atchaya Chandran R(192521394) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

14.08.2026 Experiment 31

Aim:

To simulate FIFO page replacement and find the page faults and page hits.

Code:

```
#include<bits/stdc++.h>
```

```
int main(){
int p[20],f[10],n,m,i,j,pos=0,found=0,
hit;
scanf("%d",&n);
for(i=0;i<n;i++)
  scanf("%d",&p[i]);

scanf("%d",&m);
for(i=0;i<m;i++)
f[i]=-1;

for(i=0;i<m;i++)
f[i]=-1;
for(i=0;i<n;i++){
  hit=0;
for(j=0;j<m;j++)
  if(f[j]==p[i])
    hit=1;
  if(!hit){
    f[pos]=p[i];
    pos=(pos+1)%m;
```

```
        fault++;
    }
}
printf("Page faults = %d", fault);
return 0;
}
```

Output:

Enter number of pages: 6
Enter page reference string: 1 2 3 1 4 5
Enter number of frames: 3

Page fault = 5

Result:

Thus, the FIFO page replacement technique was successfully simulated and the page faults were calculated.

Experiment – 32

14.08.2026

Aim:

To simulate Least Recently Used (LRU) replacement and find page faults.

Code:

```
#include <stdio.h>
int main () {
  int p[20], f[20], t[20], n, m, i, j;
pos, fault = 0, hit;
  scanf ("%d", &n);
  for (i=0; i<n; i++) scanf ("%d",
&p[i]);
  scanf ("%d", &m);
  for (i=0; i<m; i++) f[i] = -1;
  for (i=0; i<n; i++) {
    hit=0;
  for (j=0; j<m, j++)
   if (f[j] == p[i]) {
   hit=1; t[j]=i; break;
 }
if (!hit) {
  fault++;
  pos = 0;
for (j=1; j<m; j++)
  if (f[j] == -1 || t[j] < t[pos])
      pos = j;
```

f[pos] = p[i];
i[pos] = i;

}

printf("Page Faults = %d\n", fault);
printf("Page Faults = %d", n - fault);
return 0;

}

Output:

Page Faults = 5
Page Hits = 1

Result:
Thus, the CRC page replacement algorithm was successfully simulated.

Experiment - 32

10.08.2026

Aim:
To simulate optimal page replacement and find page faults.

Code:

#include <stdio.h>

int main() {

```
int p[10], f[10], n, m, i, j, k, bit, fault = 0;
Scanf("%d", &n);
for(i=0; i<n; i++) Scanf("%d", &p[i]);
Scanf("%d", &m);
for(i=0; i<m; i++) f[i] = -1;
for(i=0; i<n; i++) {
  bit = 0;
  for(j=0; j<m; j++)
    if(f[j] == p[i]) bit = 1;
  if(!bit) {
    f[fault++];
    pos = 0; fal = -1;
    for(j=0; j<m; j++) {
      for(k=i+1; k<n; k++)
        if(f[j] == p[k]) break;
      if(k > fal) {
        fal = k;
        pos = j;
```

```
    }
    }
    f[pos] = p[x];
    }
}
printf("Page faults = %d\n", fault);
printf("Page hits = %d\n", n - fault);
return 0;
}
```

Output:

Page faults = 5
Page Hits = 1

Result:

Thus the optimal page replacement algorithm was successfully simulated.

Experiment - 14

11.08.2026

Aim:

To simulate sequential file allocation using C.

Code:

```
#include <stdio.h>
int main()
{
int n, i, key;
printf("Enter number of records: ");
scanf("%d", &n);
int record[n];
printf("Enter records: \n");
for (i=0; i<n; i++)
  scanf("%d", &record[i]);

printf("Enter record to access: ");
scanf("%d", &key);

printf("Record found: ");
for(i=0; i<n; i++) {
printf("%d ", record[i]);

if (record[i] == key)

break;

}

if (i == n)
  printf("\n record not found");
```

return 0;
}

Output

Enter number of locations: 5

Enter locations:

10 20 30 40 50

Enter location to access: 40

Location 40 found : (0 2 0 3 0 4 0

Result:

Thus, Sequential File allocation was successfully implemented.

17.01.2026 Experiment-35

Aim:

To simulate the Indexed File Allocation strategy using a C program.

Code:

```
#include <stdio.h>
int main() {
int n, indexBlock, blocks[50];
printf("Indexed File Allocation Strategy\n");
printf("Enter the index block number: ");
```

```
scanf("%d", &indexBlock);
printf("Enter the number of file blocks: ");
scanf("%d", &fblocks);
printf("Enter the block numbers: \n");
for(int i=0; i<n; i++) {
    printf("Block %d: ", i+1);
    scanf("%d", &fileBlocks[i]);
}
printf("\nFile Allocation Table\n");
printf("-------------\n");
printf("Index Block : %d\n", indexBlock);
printf("File Block \t Disk Block\n");
return 0;
}
```

Output:

Enter Index Block: 10
Enter Number of Blocks: 5
Enter Block numbers:
21
45
18
30
56

Index Block = 20

File Block	Disk Block
0	45
1	18
2	30
3	56
4	

Result:

The Indexed File Allocation Strategy was successfully simulated using C.

17.08.2026 Experiment - 36

Aim:

To simulate the indexed file Allocation Strategy using a C program.

Code:

```
#include<stdio.h>
int main()
{
  int n, block[20];
  printf("Enter number of blocks:");
  scanf("%d",&n);
  printf("Enter block numbers:\n");
  for(int i=0;i<n;i++)
    scanf("%d",&block[i]);
```

```
printf("\n Linked Allocation:-\n");
for (int i=0; i<n-1; i++)
  printf("%d->%d\n", block[i], block[i+1]);
  printf("%d->NULL\n", block[n-1]);
  return 0;
}
```

Output:

```
Enter Number of Blocks : 5
Enter Block numbers:
  10
  25
  18
  40
  50
Linked Allocation:
10->25
25->18
18->40
40->50
50->NULL
```

Result:

The Linked File Allocation Strategy is successfully simulated using C program

17.05.24

Aim:
To simulate the FCFS disk scheduling algorithm using a C program.

Code:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int n, head, req[20], seek = 0;
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request queue: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &req[i]);
    printf("Enter initial head position: ");
    scanf("%d", &head);
    for (int i = 0; i < n; i++) {
        seek += abs(req[i] - head);
        head = req[i];
    }
    printf("Total seek time = %d\n", seek);
    return 0;
}
```

Output:

Enter number of requests : 5
Enter request queue : 98 183 37 122 14
Enter initial head position : 53
Total seek time = 640

Result:

The C++ Disk Scheduling algorithm was successfully simulated using C.

19-08-2026 Experiment - 38

Aim:

To simulate the SCAN Disk Scheduling algorithm using a C program

Code:

```
#include <stdio.h>
#include <stdlib.h>

int main () {

    int n, head, diff, seek = 0, req[20];

    printf ("Enter number of requests: ");
    scanf ("%d", &n);
```

```
printf("Enter requests: ");
for (int i = 0; i < n; i++)
    scanf("%d", &request[i]);
printf("Enter disk size: ");
scanf("%d", &disk_size);
seek = abs(cyl - request[0]);
printf("Total seek time = %d\n", seek);
return 0;
}
```

Output:

Enter number of requests: 5
Enter requests: 98 183 37 122 14
Enter initial head position: 53
Enter disk size: 200
Total seek time = 146

Result:

Thus SCAN Disk Scheduling algorithm was successfully simulated using C.

Experiment - 39

19.02.2024

Aim:
To simulate the C-SCAN Disk scheduling algorithm using a C program.

Code:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, head, disk, seek;
    printf("Enter number of requests: ");
    scanf("%d", &n);

    printf("Enter initial head position: ");
    scanf("%d", &head);

    printf("Enter disk size: ");
    scanf("%d", &disk);

    seek = (disk - 1 - head) + (disk - 1);
    printf("Total seek time = %d\n", seek);
    return 0;
```

Input:
Total number of requests: 5
Total initial head position: 53
Total disk size: 200
Total seek time = 345.

Result:
C-SCAN disk scheduling algorithm was successfully simulated using C.

Experiment - 40

14-08-2026

Aim:
To illustrate the various file access permissions and different types of links.

Commands:

ls -l
chmod 755 file.txt
chmod u+rwx file.txt
chmod g+rw file.txt
chmod o+r file.txt
chown user file.txt
chgrp group file.txt

Output:

```
-rwxr-xr-x 1 ubit ubit 120 Aug 14 10:
file.out
```

Result:

The program file created successfully and different types of cells in Linux were displayed successfully.